

Biyomedikal İşaret ve Görüntü İşleme Ödev 6: Toplamsal Histogram

Muhammed Kayra Bulut - 25501805

Kasım 2025

1 Giriş

Bu ödevde gri seviye tıbbi görüntüler üzerinde histogram analizi yapılmıştır. Önce her görüntü için normal ve toplamsal (kümülatif) histogramlar hesaplanmış, daha sonra lineer histogram eşitleme uygulanmıştır. Son aşamada ise her görüntünün histogramı, farklı referans görüntülere göre hem manuel histogram specification yöntemiyle hem de hazır fonksiyon kullanılarak histogram matching ile yeniden düzenlenmiştir.

2 Sonuçlar

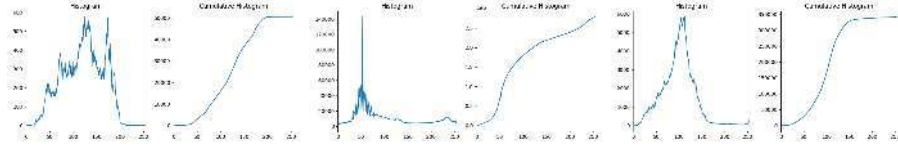
Aşağıdaki alt bölümlerde her bir adımda elde edilen temel çıktılar özetlenmiştir.

2.1 Orijinal Görüntüler



Şekil 1: Orijinal gri seviye görüntüler (gray_scale_1, gray_scale_2, gray_scale_3).

2.2 Histogram ve Toplamsal Histogramlar (a Şikkı)



Şekil 2: gray_scale_1, gray_scale_2 ve gray_scale_3 için histogram (solda) ve toplamsal histogram (sağda) grafikleri.

2.3 Lineer Histogram Eşitleme Sonuçları (b Şikkı)



Şekil 3: Lineer histogram eşitlemesi uygulanmış görüntüler. Kontrast dağılımının daha dengeli hale geldiği gözlenmektedir.

2.4 Histogram Specification (c Şikkı)

Aşağıdaki şekilde her bir kaynak görüntünün, diğer iki görüntüye referans alınarak manuel histogram specification ile dönüştürülmüş halleri gösterilmektedir.



Şekil 4: Her görüntünün diğer görüntüler referans alınarak manuel histogram specification ile dönüştürülmüş halleri.

2.5 Hazır Fonksiyon ile Histogram Matching (d Şıkkı)

Bu adımda `histogram_matching_cv` fonksiyonu kullanılarak benzer bir histogram uydurma işlemi yapılmıştır.



Şekil 5: Hazır fonksiyon benzeri yapı ile elde edilen histogram matching sonuçları. Manuel specification sonuçlarıyla büyük oranda benzer görünmektedir.

3 Genel Değerlendirme

Histogram eşitleme ile özellikle düşük kontrastlı görüntülerde ayrıntıların daha net hale geldiği görülmüştür. Histogram specification ve matching adımları ile farklı görüntülerin parlaklık ve kontrast dağılımlarının birbirine uydurulabildiği gözlemlenmiştir. Bu tür işlemler, tıbbi görüntülerde farklı cihaz veya çekim koşullarını normalize etmek için kullanılabilir.

4 Kod

Kod aşağıda verilmiştir:

```
import os
from pathlib import Path

import cv2
import numpy as np
from matplotlib import pyplot as plt

BASE_DIR = Path(__file__).resolve().parent
SRC_DIR = BASE_DIR / "src"
```

```

DEST_DIR = BASE_DIR / "dest"

def ensure_dest_dir() -> None:
    DEST_DIR.mkdir(exist_ok=True)

def load_gray_images(src_dir: Path):
    images = {}
    for name in sorted(os.listdir(src_dir)):
        path = src_dir / name
        if not path.is_file():
            continue
        img = cv2.imread(str(path), cv2.IMREAD_GRAYSCALE)
        if img is None:
            continue
        images[name] = img
    return images

def compute_histogram(image: np.ndarray, bins: int = 256):
    hist, bin_edges = np.histogram(image.flatten(), bins
    =bins, range=(0, 256))
    return hist, bin_edges

def compute_cumulative_histogram(hist: np.ndarray):
    return hist.cumsum()

def save_hist_and_cumulative(name: str, hist: np.ndarray,
    cum_hist: np.ndarray):
    plt.figure(figsize=(8, 4))
    plt.subplot(1, 2, 1)
    plt.title("Histogram")
    plt.plot(hist)
    plt.xlim([0, 255])

    plt.subplot(1, 2, 2)
    plt.title("Cumulative Histogram")
    plt.plot(cum_hist)
    plt.xlim([0, 255])

    out_path = DEST_DIR / f"{name}_hist_cum.png"
    plt.tight_layout()
    plt.savefig(out_path)
    plt.close()

```

```

def linear_hist_equalization(image: np.ndarray):
    return cv2.equalizeHist(image)

def save_image(name: str, image: np.ndarray, suffix: str):
    out_path = DEST_DIR / f"{name}_{suffix}.png"
    cv2.imwrite(str(out_path), image)

def histogram_specification(source: np.ndarray, reference:
    np.ndarray):
    # manual histogram specification / matching (
    grayscale)
    src_hist, _ = np.histogram(source.flatten(), 256,
        [0, 256])
    ref_hist, _ = np.histogram(reference.flatten(), 256,
        [0, 256])

    src_cdf = src_hist.cumsum().astype(np.float64)
    ref_cdf = ref_hist.cumsum().astype(np.float64)

    src_cdf /= src_cdf[-1]
    ref_cdf /= ref_cdf[-1]

    lookup = np.zeros(256, dtype=np.uint8)
    ref_idx = 0
    for src_gray in range(256):
        while ref_idx < 255 and ref_cdf[ref_idx] <
            src_cdf[src_gray]:
            ref_idx += 1
        lookup[src_gray] = ref_idx

    matched = lookup[source]
    return matched

def histogram_matching_cv(source: np.ndarray, reference: np.
    ndarray):
    return histogram_specification(source, reference)

def main():
    ensure_dest_dir()

    images = load_gray_images(SRC_DIR)
    if len(images) < 3:
        raise RuntimeError("src dizininde en az 3
            gri seviye goruntu olmalı")

    # a) Her goruntu icin histogram ve toplamsal

```

```

        histogram
    for name, img in images.items():
        hist, _ = compute_histogram(img)
        cum_hist = compute_cumulative_histogram(hist)
        save_hist_and_cumulative(Path(name).stem,
                                hist, cum_hist)

    # b) Linear histogram eşitleme
    equalized_images = {}
    for name, img in images.items():
        eq = linear_hist_equalization(img)
        equalized_images[name] = eq
        save_image(Path(name).stem, eq, "equalized")

    # c) Histogram specification: her görüntüyü farklı
        referanslara göre
    names = list(images.keys())
    n = len(names)
    for i, src_name in enumerate(names):
        src_img = images[src_name]
        # referans olarak diğer görüntüleri kullan
        for j, ref_name in enumerate(names):
            if i == j:
                continue
            ref_img = images[ref_name]
            matched = histogram_specification(
                src_img, ref_img)
            base = f"{Path(src_name).stem}_to_{Path(ref_name).stem}_spec"
            save_image(base, matched, "img")

    # Aynı eşleşmeyi "cv" isimli çıktılarıyla tekrar
        kaydedelim
    for i, src_name in enumerate(names):
        src_img = images[src_name]
        for j, ref_name in enumerate(names):
            if i == j:
                continue
            ref_img = images[ref_name]
            matched_cv = histogram_matching_cv(
                src_img, ref_img)
            base = f"{Path(src_name).stem}_to_{Path(ref_name).stem}_matchcv"
            save_image(base, matched_cv, "img")

if __name__ == "__main__":
    main()

```

Listing 1: main.py